

Topic 13: Graphs and Graph Traversal

13.1 Graph Terminology

- Graph $G = (V, E)$: set of vertices V connected by edges E
- Directed graph: edges have direction ($A \rightarrow B$ does not imply $B \rightarrow A$)
- Undirected graph: edges are bidirectional
- Weighted graph: each edge has a numerical weight
- Path: sequence of vertices connected by edges
- Cycle: path that starts and ends at the same vertex
- Connected: every vertex reachable from every other vertex (undirected)

13.2 Graph Representations

- Adjacency Matrix: $V \times V$ 2D array -- $\text{matrix}[u][v]=1$ if edge exists -- $O(V^2)$ space
- Adjacency List: array of lists -- $\text{list}[u]$ contains all neighbours of u -- $O(V+E)$ space
- Adjacency list preferred for sparse graphs; matrix for dense graphs or $O(1)$ edge queries

```
// Adjacency list using vector of vectors
int V = 5;
vector<vector<int>> adj(V);

// Add undirected edge between u and v
void addEdge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}
```

13.3 Breadth-First Search (BFS)

- Visit all neighbours of a vertex before going deeper -- level by level
- Use a queue and a visited array
- Time: $O(V+E)$, Space: $O(V)$
- Applications: shortest path in unweighted graph, level-order tree traversal, connected components

```

void BFS(int start) {
    vector<bool> visited(V, false);
    queue<int> q;
    visited[start] = true;
    q.push(start);
    while (!q.empty()) {
        int v = q.front(); q.pop();
        cout << v << " ";
        for (int u : adj[v])
            if (!visited[u]) { visited[u]=true; q.push(u); }
    }
}

```

13.4 Depth-First Search (DFS)

- Explore as far as possible along each branch before backtracking
- Use recursion (implicit stack) or an explicit stack
- Time: $O(V+E)$, Space: $O(V)$
- Applications: cycle detection, topological sort, connected components, maze solving

```

vector<bool> visited(V, false);

void DFS(int v) {
    visited[v] = true;
    cout << v << " ";
    for (int u : adj[v])
        if (!visited[u]) DFS(u);
}

```